Peripheral Interrupt Experiment

1. Peripheral Interrupt Overview

The previous experiment used **polling** to detect the button — the CPU continuously reads the GPIO state. Polling is simple but inefficient: the CPU idles most of the time. **Interrupts** are another paradigm: the CPU executes the main loop normally, and when the GPIO level changes, the hardware automatically pauses the current code, jumps to the ISR (Interrupt Service Routine) to handle the event, then returns to the breakpoint and continues.

External Interrupt (EXTI) is the hardware mechanism by which an MCU responds to external asynchronous events. When the GPIO pin detects a specified edge (rising edge / falling edge / any edge), the hardware automatically pauses the current program execution, jumps to the preset **Interrupt Service Routine (ISR)** to handle the event, and returns to the original program to continue after completion.

This experiment demonstrates GPIO external interrupts: the falling edge of the BOOT button (GPIO0) triggers the interrupt, the ISR sets a flag, and the main loop toggles the LED after detecting the flag.

Polling vs Interrupt comparison:

Comparison	Polling	Interrupt
CPU usage	Continuously polls, consuming CPU even when nothing happens	CPU is idle when nothing happens
Response latency	Depends on the polling interval	Microsecond-level (triggered directly by hardware)

Comparison	Polling	Interrupt
Complexity	Simple	ISR has constraints (no delay/print/memory allocation)
Applicable scenario	Simple local control	Low power consumption, real-time response

---

2. Core Concepts

2.1 Interrupt Trigger Modes

Trigger Mode	`Pin` Constant	Description
Falling edge	`Pin.IRQ_FALLING`	Triggered at the high→low moment
Rising edge	`Pin.IRQ_RISING`	Triggered at the low→high moment
Both edges	`Pin.IRQ_RISING | Pin.IRQ_FALLING`	Triggered on any change
Low level	`Pin.IRQ_LOW_LEVEL`	Continuously triggered while low (rarely used)
High level	`Pin.IRQ_HIGH_LEVEL`	Continuously triggered while high (rarely used)

> The button experiment uses `Pin.IRQ_FALLING`: pressing the button (1→0) triggers once, releasing does not trigger.

2.2 ISR Design Principles

MicroPython ISRs run in the hardware interrupt context and have the following constraints:

Allowed	Forbidden
Set a flag (`global flag = True`)	`time.sleep()` / `time.sleep_ms()`
Simple assignments	`print()` / memory allocation
`Pin.value()` reads	Creating objects
—	Long loops

> **Golden rule:** the ISR only sets a flag, and the main loop does the actual processing. `print()` inside the ISR may cause the board to crash.

2.3 Software Debouncing

The interrupt captures the moment of level change, but it may still trigger multiple times in succession due to bounce. The handling approach is the same as polling: **ISR sets a flag → main loop delays 20ms → double-check**.

2.4 Core API

```
key.irq(trigger=Pin.IRQ_FALLING, handler=key_isr)
```

Parameter	Description
`trigger`	Trigger condition: `IRQ_FALLING` / `IRQ_RISING` / combination
`handler`	ISR callback function, receives one argument `pin` (the triggering source)

3. Hardware Dependencies

GPIO	Peripheral	Mode	Description
0	KEY (BOOT)	`Pin.IN` + `Pin.PULL_UP`, falling-edge interrupt	Pressed=0
44	LED0	`Pin.OUT`	Onboard LED, common-anode, `0`=on

4. Project Architecture and Flow

```
Script execution (single-file .py)
  |
  ├─ led = Pin(44, Pin.OUT, value=0)              // LED initially on (common-anode: 0=on)
  ├─ key = Pin(0, Pin.IN, Pin.PULL_UP)            // button input + pull-up
  ├─ led_state = False, key_flag = False          // global state (led_state=False → 0=on)
  ├─ def key_isr(pin): key_flag = True            // ISR: only sets a flag
  ├─ key.irq(trigger=FALLING, handler=key_isr)    // bind the interrupt
  |
  └─ while True:                                  // main loop
       ├─ if key_flag:                             // detect flag
       |    ├─ key_flag = False                    // clear flag
       |    ├─ sleep_ms(20)                        // debounce
       |    ├─ if key.value() == 0:                // double-check
       |    |    ├─ led_state ^= 1
       |    |    └─ led.value(led_state)
       └─ sleep_ms(10)                             // idle, reduce usage
```

5. Source Code Details

Full source code: `5.Source Code → MicroPython → 1.Basic Peripherals → 3.KEY_NVIC → 3.KEY_NVIC.py`

5.1 Pins and Global Variables

```
from machine import Pin  # Pin class: GPIO control
import time              # delay functions

LED_PIN = 44  # onboard LED
KEY_PIN = 0   # BOOT button

led_state = False  # LED state, initially on (0=on)
key_flag = False   # interrupt flag
```

5.2 Interrupt Service Routine (ISR)

```
led = Pin(LED_PIN, Pin.OUT, value=0)      # LED default on, common-anode: 0=on
key = Pin(KEY_PIN, Pin.IN, Pin.PULL_UP)   # button input + pull-up

def key_isr(pin):  # interrupt callback
    global key_flag
    key_flag = True  # only set the flag, no blocking operations

key.irq(trigger=Pin.IRQ_FALLING, handler=key_isr)  # falling edge trigger
```

> The ISR only does `key_flag = True`; it does not delay, print, or toggle the LED inside.

5.3 Main Loop — Flag Handling + Debounce

```
while True:
    if key_flag:                    # interrupt fired
        key_flag = False            # clear flag
        time.sleep_ms(20)           # debounce
        if key.value() == 0:        # confirm the button is still pressed
            led_state = not led_state  # toggle LED state
            led.value(led_state)

    time.sleep_ms(10)  # main loop idle, reduce CPU usage
```

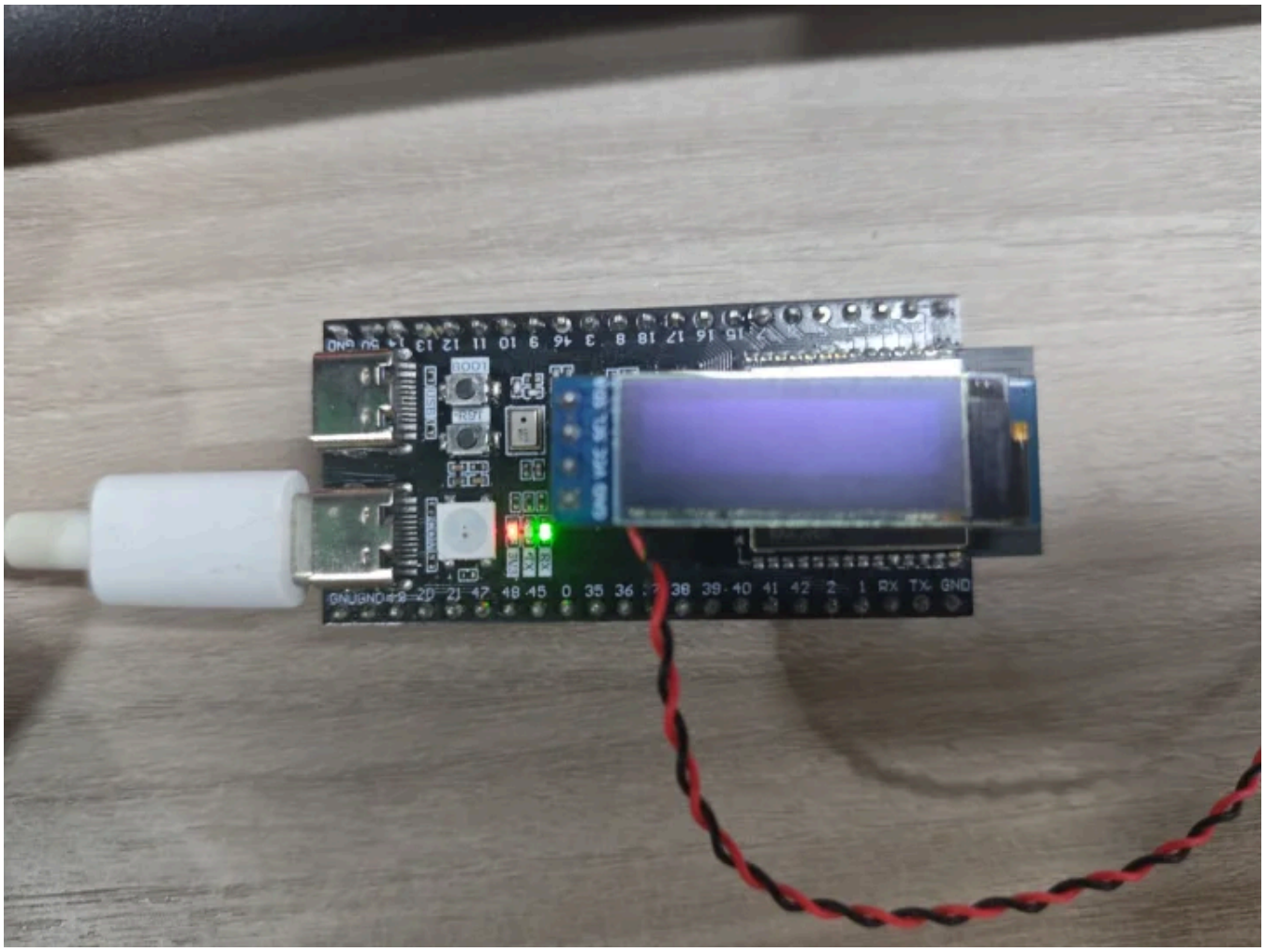
---

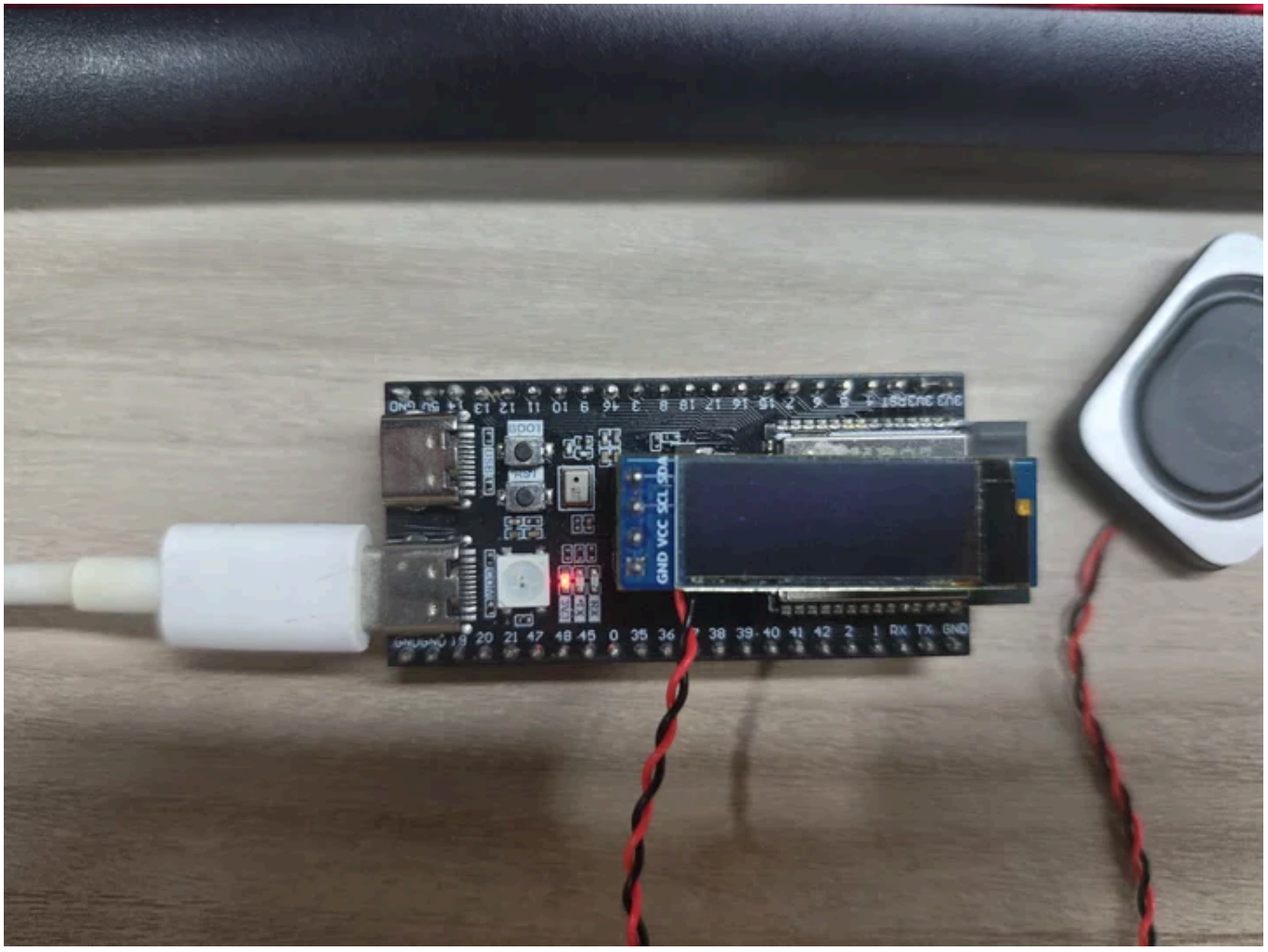
6. Operation Procedure

6.1 Flashing and Running

Thonny IDE steps: See the `1. Before Development → 3.Thonny IDE` chapter.

6.2 Normal Phenomenon





- After power-up, the LED is on and the CPU is idle (no busy-waiting polling loop)
- Press BOOT once → LED off
- Press again → LED on

6.3 Verification Methods

Operation	Expected Result
Power up	LED is on
Press BOOT once	LED is off
Press again	LED is on
Add `print("ISR")` in the ISR	May crash (no print in ISR)
Comment out `key_flag = False`	After pressing once, the LED toggles every 10ms (flag not cleared)

7. Common Problems

Symptom	Cause	Solution
Pressing once toggles the LED repeatedly	Bounce causes multiple interrupts	Add debouncing when handling the flag
No response after multiple presses, suddenly triggers once	ISR constraints not followed, stack overflow	Check whether the ISR does delay/print/object creation
Interrupt does not trigger	`irq()` parameters wrong or handler misspelled	Confirm `trigger=Pin.IRQ_FALLING` and the handler function name
`led_state` changed but LED unchanged	`global led_state` not declared	Ensure the `global` declaration exists in the ISR or main loop